How the Apposto Delivery Bot Works

A plain-language guide, explained through one real order

For: the owner of @Apposto_Delivery_Bot

Written: 8 November 2026

How to read this guide

Read **Part 1** first. It explains the whole system in one page.

Part 3 is the heart of the guide: one order, followed step by step from the customer's first message to the delivery.

You can come back to the other parts when you need them. You do not need to understand the code to run the business.

Part 1 · The whole system on one page

Your business connects three groups of people: **customers** who are hungry, **shops** that cook food, and **riders** on bicycles who carry it. Before, you would have had to phone everyone. Now the **bot** does the talking for you inside Telegram, and it writes everything down in a **Google Sheet**.

The people (the "cast")

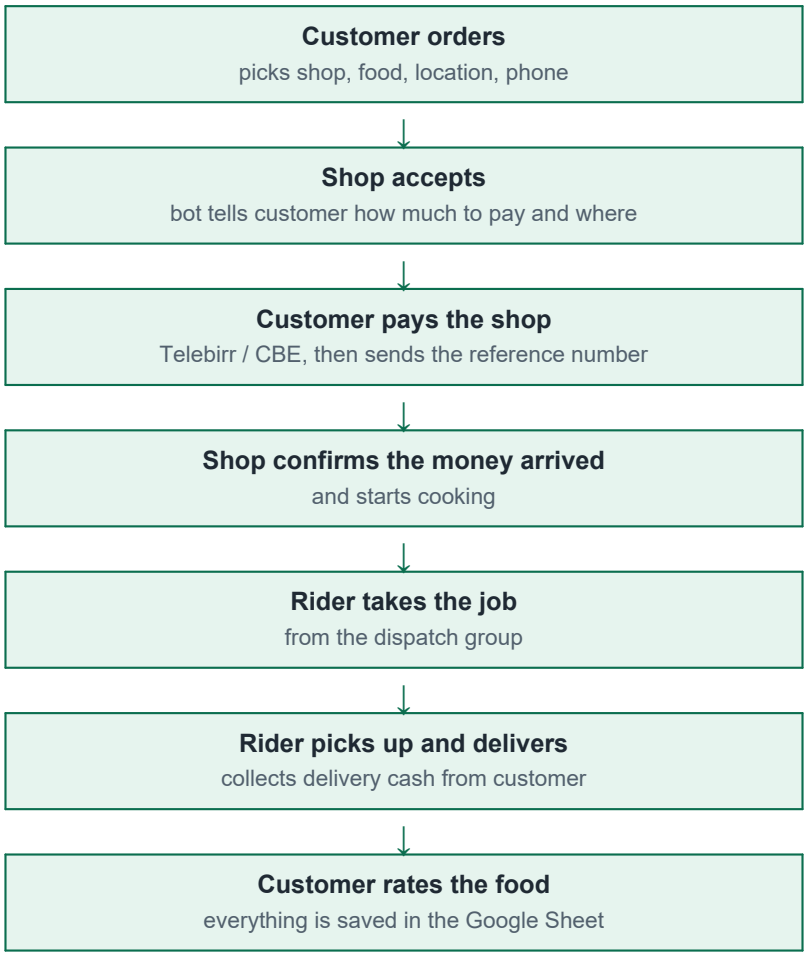
Who	What they do	How they use the bot
Customer	Orders food, pays the shop by Telebirr or CBE, pays the rider cash for delivery	Private chat with the bot. Presses buttons, shares location, types phone number and payment reference
Shop (merchant)	Accepts the order, checks the money arrived, cooks, hands food to the rider	Gets messages with buttons: Accept / Reject, Received / Not received, Ready for pickup
Rider	Takes the job, picks up the food, delivers it, collects cash	Sees jobs in the dispatch group, taps "I'll take it", then gets details privately
You (admin)	Watch everything, step in when something goes wrong, collect commission	Member of the dispatch group, plus admin commands like /today, /active, /cancel

The machines (the "stage")

Part	What it is	Its job
Telegram	The chat app everyone already has	Carries messages and button presses between people and the bot
The bot program	The code in this project (Node.js), running on Render	The "brain". Decides what to reply, what buttons to show, who to notify
Render	A website that rents out computers on the internet	Keeps the bot program running 24 hours a day, so your PC can be off
Google Sheet	A normal spreadsheet	The "notebook". Holds shops, menus, riders, customers, every order and every review
Dispatch group	A Telegram group: you plus all riders	The "control room". The bot posts every order and every job there

> **In plain words**
>
> Telegram is the telephone line. The bot program is a receptionist sitting on Render's computer. The Google Sheet is the receptionist's notebook. The dispatch group is the noticeboard in the back room where riders wait for jobs.

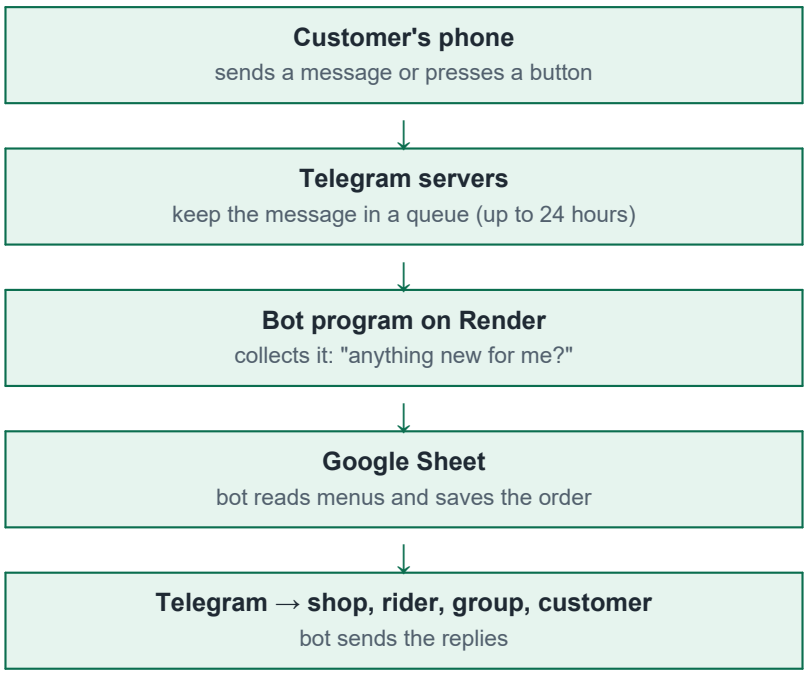
The shortest possible story of an order



Part 2 · How the pieces talk to each other

How the bot "hears" messages

The bot does not wait for Telegram to call it. Instead, it calls Telegram again and again, all day, asking *"Is there anything new for me?"*. This is called **long polling**. When a customer presses a button, Telegram holds that message until the bot asks for it, then hands it over. The bot reads it, decides what to do, writes to the Google Sheet if needed, and sends replies back through Telegram.



Why the bot went silent on 8 November (and how it was fixed)

On Render's **free** plan, the computer goes to sleep after **15 minutes** if nobody visits its web address. The bot's "anything new?" questions go *out* to Telegram, so Render does not count them as visits. After 15 quiet minutes, the bot fell asleep and messages piled up in Telegram (3 were waiting).

When someone opened **https://delivery-bot-ezum.onrender.com/health**, Render woke the bot up (it took about 26 seconds) and it answered all waiting messages at once.

Permanent fix: a free service such as UptimeRobot visits that /health address every 5 minutes, so the bot never sleeps.

Only one copy of the bot may run

Telegram only lets **one** program collect messages for a bot. If the bot runs on Render *and* on your PC (with **npm start**) at the same time, they fight. You will see *"409 Conflict: terminated by other getUpdates request"* in the logs and messages go randomly to one or the other. Rule: **when Render is running, do not run the bot on your PC.**

Secrets and settings (environment variables)

The bot needs some private keys to work. These are not stored in the code, because the code is on GitHub. On your PC they live in the **.env** file. On Render they are typed into the service's **Environment** page.

Setting	What it is	If it is wrong
BOT_TOKEN	The bot's password, from @BotFather	Bot cannot connect to Telegram at all
GOOGLE_SHEET_ID	Which spreadsheet to use (from its web address)	Bot cannot find the shops and menus
google-key.json	The Google "robot account" key. On Render it is a Secret File	Bot cannot read or write the sheet

Setting	What it is	If it is wrong
DISPATCH_CHAT_ID	The dispatch group's id (starts with -100)	Orders and jobs reach nobody
ADMIN_IDS	Your Telegram id (and your partner's)	You cannot use /today, /active, /cancel or press buttons for shops
SUPPORT_PHONE	Phone number shown to customers	Customers have no number to call

Prices and limits also have settings (delivery fee, maximum distance, commission and so on). They have sensible defaults, listed in Part 4.

Updating the bot

When code changes are pushed to GitHub (**git push**), Render notices, rebuilds and restarts the bot by itself in about a minute. Shops, menus and riders are **not** in the code: you change them directly in the Google Sheet, and the bot picks up the change within 60 seconds (or at once with **/reload**).

Part 3 · One order, step by step

Let us follow a real-looking order. The names are examples.

Role	Example
Customer	**Abebe**, at home near the stadium, about 2 km from the shop in a straight line
Shop	**Hamarawi Hotel** (shop id M1 in the sheet). Burger costs 250 Br
Rider	**Kebede**, listed in the Riders tab as active
You	Admin, sitting in the dispatch group

Step 1 · Abebe opens the bot — Customer

Abebe taps the bot link and presses **Start**. The bot asks him to choose a language: Afaan Oromo, Amharic or English. It remembers his choice for next time.

He sees a menu with buttons: *Order*, *My orders*, *Help*, *Language*.

Step 2 · He picks a shop and food — Customer

He taps **Order**. The bot reads the **Merchants** tab and shows only shops marked active. He picks Hamarawi Hotel.

The bot reads the **Menu** tab and shows that shop's items with + and – buttons. He adds **2 burgers**. The cart shows 500 Br.

If he wants something not on the menu, he can type a **custom request** (for example "one fresh mango juice"). The shop will set the price later.

He taps **Checkout**.

Step 3 · Location, landmark, phone — Customer

Location: Abebe shares his GPS location. The bot measures the distance from the shop (the shop's lat/lng in the sheet) to him. It takes the straight line, 2.0 km, and multiplies by 1.3 because roads are not straight: about **2.6 km**.

The bot uses the distance to set the delivery fee: **90 Br** (see Part 4). If the distance were over **3 km**, the bot would politely refuse, because bicycles cannot go that far.

Landmark: he types "Blue gate behind the stadium". This helps the rider find him.

Phone: he types 0911 223344. The bot checks it is a real Ethiopian mobile number (09… or 07…).

Step 4 · He confirms the order — Customer

The bot shows a summary: 2 burgers 500 Br, delivery 90 Br, service fee 10 Br. He taps **Place order**.

The bot gives the order an id like **261108-K7QM** (date, then 4 letters that are easy to say on the phone; it never uses 0, O, 1 or I). It writes a new row in the **Orders** tab.

The shop gets a message: *"New order: 2 x Burger, Blue gate behind the stadium"* with **Accept** and **Reject** buttons.

The dispatch group gets a copy, with a **Cancel (admin)** button for you.

Sheet status becomes: PLACED

Step 5 · The shop accepts — Shop

Hamarawi Hotel taps **Accept**. (If Abebe had typed a custom request, the bot would first ask the shop to type the food price.)

The bot works out your commission: 8% of 500 = **40 Br**, and saves it on the order.

Abebe gets: *"Pay 500 Br to the shop: Telebirr 09…, account name …. Then send the transaction reference number here. Pay 100 Br (delivery + service) to the rider in cash when it arrives."*

Sheet status becomes: ACCEPTED

Step 6 · Abebe pays the shop and sends the reference — Customer

Abebe pays 500 Br by Telebirr **directly to the shop's account**. Your business never touches the food money.

He copies the reference from the Telebirr SMS (for example CK12AB34CD) and sends it to the bot.

The bot checks the reference looks real (6–25 letters and digits) and that **no other order already used it**. Then the shop gets: *"Check your account: 500 Br, ref CK12AB34CD"* with **Received** / **Not received** buttons.

Sheet status becomes: PAYMENT_SUBMITTED

Step 7 · The shop confirms the money really arrived — Shop

The shop looks at **its own** Telebirr account. The money is there, so it taps **Received**. This is the main protection against fake payment screenshots: the shop never cooks before the money is really in its account.

Abebe gets: *"Payment confirmed. Your food arrives in about 35–45 min."*

The shop gets a **Ready for pickup** button for later.

The dispatch group gets the job: *"New delivery 261108-K7QM. Pickup: Hamarawi Hotel. Drop: Blue gate behind the stadium, 2.6 km. Rider pay: 68 Br"* with an **I'll take it** button.

Sheet status becomes: CONFIRMED

Step 8 · Kebede takes the job — Rider

Kebede taps **I'll take it** in the group. The bot checks that he is in the **Riders** tab and active. Strangers in the group cannot take jobs.

If two riders tap at the same second, only one gets it. The other sees "Already taken".

Kebede gets a **private** message with everything: shop name and phone, customer name and phone, landmark, the cash to collect (100 Br), his pay (68 Br), and a map pin of Abebe's location.

The group message changes to "Taken by Kebede". Abebe and the shop are told the rider's name and phone.

Step 9 · The food is ready — Shop

When the food is ready the shop taps **Ready for pickup**. Kebede gets a message to come and collect it.

Sheet status becomes: READY

Step 10 · Pickup — Rider

At the shop Kebede taps **Picked up**. Abebe gets: *"Your order is on the way. Have 100 Br cash ready. About 15–20 min."* The shop is told the food left.

Sheet status becomes: PICKED_UP

Step 11 · Delivery Rider

Kebede hands over the food and collects **100 Br cash**. He taps **Delivered**. The bot tells him "you earned 68 Br".

The dispatch group sees: *"261108-K7QM delivered by Kebede · you earned 72 Br"*.

Sheet status becomes: DELIVERED

Step 12 · Rating Customer

Abebe is asked to rate the food from 1 to 5 stars, then give the rider a thumbs up or down, then optionally write a comment. The review goes into the **Reviews** tab and the shop is told.

A low rating (1–2 stars) or a thumbs down is also posted in the dispatch group, so you can follow up.

A shop's average rating is only shown to customers once it has at least 5 reviews, to be fair to new shops.

The order's life, in one line

PLACED → ACCEPTED → PAYMENT_SUBMITTED → CONFIRMED → READY → PICKED_UP → DELIVERED

An order can also end as **REJECTED** (shop said no) or **CANCELLED**. The bot refuses any step out of order, so for example a payment can never be confirmed twice, and an old button pressed later does nothing harmful.

Part 4 · Where the money goes

There are two separate payments, and the business does not hold the customer's food money:

- **Food money** goes from the customer straight to the **shop**, by Telebirr or CBE.
- **Delivery fee + service fee** go from the customer to the **rider**, in cash.
- Your **commission** (8% of the food) is written on every order. You collect it from each shop at the end of the month.

Abebe's order in numbers

Item	How it is calculated	Amount
Food	2 burgers × 250 Br	500 Br
Distance	Straight line 2.0 km × 1.3 (road factor)	2.6 km
Delivery fee	60 Br covers the first 1.5 km. 1.1 km is left over, counted as 2 started km × 15 Br = 30 Br	90 Br
Service fee	Fixed per order	10 Br
Customer pays the shop	Food	500 Br
Customer pays the rider (cash)	Delivery 90 + service 10	100 Br
Rider's pay	75% of the delivery fee (67.5, rounded)	68 Br
Commission (shop owes you)	8% of 500 Br, collected monthly	40 Br
What the business earns	Commission 40 + service 10 + delivery 90 – rider pay 68	**72 Br**

Who holds which money after the delivery

Shop: has 500 Br, but owes you 40 Br commission at month end.

Rider: holds 100 Br cash. Keeps 68 Br, owes you 32 Br.

You: 40 Br (from the shop, monthly) + 32 Br (from the rider) = 72 Br.

The bot records these numbers in the sheet, but it does not move money. Collecting cash from riders and invoicing shops is done by you.

The default price settings

Setting	Default	Meaning
DELIVERY_FEE_BASE	60 Br	Price for the first BASE_KM
BASE_KM	1.5 km	Distance covered by the base price
PER_KM_FEE	15 Br	Added for each started km after that
MAX_KM	3 km	Further than this, the order is refused (bicycles)
ROAD_FACTOR	1.3	Straight line × 1.3 ≈ real road distance
SERVICE_FEE	10 Br	Fixed fee per order
RIDER_SHARE_PCT	75%	Rider's share of the delivery fee

Setting	Default	Meaning
DEFAULT_COMMISSION_PCT	8%	Used when the shop row has no commission_pct

If a customer skips sharing the location, the bot cannot measure the distance. It charges the base fee (60 Br) and the rider calls the customer to find them.

How the delivery time is guessed

Time = shop's cooking time + 5 min for the rider to reach the shop + 4.5 min per km of riding + 5 min spare. For Abebe: 20 + 5 + 12 + 5 ≈ 42 min, shown as a range: **35–45 min**. After pickup, only the ride is left: **15–20 min**.

Cooking time starts at 20 minutes (or the shop's prep_min in the sheet). After a shop has 5 delivered orders, the bot **learns** its real cooking time from the gap between "payment confirmed" and "ready" on its last orders.

Part 5 · When things do not go to plan

Each situation below is something that will happen in real life, and what the bot does.

Situation	What happens
The shop says no	The shop taps **Reject** (for example, out of burgers). Abebe is told, the group is told, and the order ends as **REJECTED**. No money has moved yet.
The customer changes their mind	Abebe can cancel from **My orders**, but **only before he sends money** (status PLACED or ACCEPTED). After that, only you can cancel, because money is involved.
The payment reference is fake	The shop checks its account, finds nothing, and taps **Not received**. The order goes back to ACCEPTED, Abebe is asked to send the correct reference, and the group gets a warning: *"Watch for fake payments"*.
Someone reuses an old reference	The bot remembers every reference. If the same number is sent for another order, it refuses: "This reference was already used".
You need to cancel an order	Use the **Cancel (admin)** button in the group, or type **/cancel 261108-K7QM**. The customer, shop and rider are all told.
Two riders tap "I'll take it" together	The bot handles one tap at a time per order. The first one wins, the second gets "Already taken".
A rider never pressed Start	Telegram does not let bots message people who never opened them. If such a rider takes a job, the bot gives the job back to the group and tells him to open the bot and press Start first.
Someone not in the Riders tab taps a job	Refused: "You are not registered as a rider". Add them to the Riders tab with active = TRUE.
The shop has no Telegram	Leave its telegram_id empty in the sheet. All its messages (new order, check payment, start cooking) go to the **dispatch group** with the shop's phone number. You phone the shop and press the buttons for them. Admins are allowed to.
A stranger presses someone else's button	Every button checks who pressed it. Only the right shop (or an admin) can accept or confirm payment, only the assigned rider can mark picked up or delivered, only the customer can pay or review their own order.
The customer is too far away	If the road distance is over 3 km, the bot apologises and stops the order before it is placed.
The bot restarts or Render falls asleep	**Placed orders are safe**, because they are in the Google Sheet. What is lost is a customer's *unfinished* cart (items chosen but not yet ordered), because carts are only kept in the program's memory. Messages sent while the bot was asleep wait in Telegram and are answered when it wakes up.
Someone types a spreadsheet formula	Text such as "=SUM(...)" typed by a user is neutralised before saving, so nobody can mess with your sheet through the bot.

Part 6 · The Google Sheet, tab by tab

The sheet is both the bot's memory and your control panel. You **write** the first three tabs by hand. The bot writes the rest.

Tab	Who writes it	What is in it
Merchants	You	One row per shop: id (M1), names in 3 languages, telegram_id, phone, language, Telebirr/CBE account and account name, commission %, location (lat/lng), active, prep_min
Menu	You	One row per item: item id, shop id, names, price, active. Keep about 5–10 top items per shop
Riders	You	One row per rider: telegram_id, name, phone, language, active
Customers	Bot	Everyone who has ordered: Telegram id, name, phone, language
Orders	Bot	Every order: id, times, status, shop, items, money, payment reference, customer, rider
Reviews	Bot	Stars, rider thumb and comment for each delivered order

Two rules for the sheet

Never sort or delete rows in the Orders tab. The bot reads an order and then writes back to the same row, so moving rows can make it update the wrong order. To look at the data a different way, use *Data → Create a filter view*.

Never change a shop's merchant_id (M1, M2…) after orders exist for it.

How to find someone's telegram_id

Ask the person (shop owner or rider) to open the bot and send **/myid**. The bot replies with their number. Put that number in the sheet. Sending /myid inside the dispatch group shows the group's id (starts with -100), which goes into DISPATCH_CHAT_ID.

Part 7 · Your daily job as admin

Command	What it shows or does
/today	Today's orders, commission, fees, rider pay and what you earned
/active	Every open order and its status, so you can spot stuck ones
/cancel ORDER-ID	Cancel an order and tell customer, shop and rider
/reload	Re-read shops, menus and riders from the sheet right now
/myid	Show your Telegram id and the chat id (anyone can use it)

A good routine:

- Keep the dispatch group on notifications. Every new order, payment and job appears there.
- If an order sits in the same status for a long time, phone the shop or rider. /active shows them.
- React to warnings in the group: "payment NOT received", low ratings, thumbs down.
- Each evening, send /today. Collect cash from riders (what they collected minus their pay).

- At month end, add up each shop's commission from the Orders tab and invoice them.

Monthly commission for shop M1 in November 2026 (put in any empty cell):
=SUMIFS(Orders!M:M, Orders!E:E, "M1", Orders!D:D, "DELIVERED", Orders!B:B, "2026-11*")

Part 8 · Before the first real order: checklist

Everything below must be true for an order to go all the way through. The notes on the right are what was known on 7–8 November 2026. Check the sheet, because you may have changed things since.

Must be true	Why	Last known
Bot running on Render and kept awake	Otherwise nobody gets replies	Running. Set up the UptimeRobot 5-minute ping
DISPATCH_CHAT_ID set, bot in the group	Orders and jobs are posted there	Done: group "Wealthy empire"
Shop row: telegram_id	So the shop gets its own buttons (or leave empty and run it from the group)	Missing for M1
Shop row: telebirr / cbe_account and account_name	Customers need to know where to pay	account_name missing for M1
Shop row: lat and lng	Needed to measure distance and price delivery	Missing for M1
At least one rider row, active = TRUE	Nobody else can take jobs	Riders tab empty
Each rider pressed Start in the bot	So the bot can send them job details	Not yet
ADMIN_IDS contains your id	For /today, /cancel and acting for shops	Check on Render
Old test orders cancelled	Keep /today and /active clean	/cancel 261106-PGZW, 261106-MMY6, 261107-G7UB

A safe full test (15 minutes)

- Use two phones: one as the **customer**, and your own as **admin + shop + rider** (put your id in the shop's telegram_id and in the Riders tab for the test).
- Order from the customer phone and follow Part 3. At each step, look at the Orders tab and watch the status change.
- Try the "bad" paths too: tap Not received, send the same reference twice, cancel an order.
- Afterwards, put the real shop's and rider's ids back in the sheet.

Part 9 · Where things live in the code (for later)

You do not need this to run the business. It is a map for when you or a developer change something.

File	What it does
src/index.js	Starts everything: reads settings, connects to the sheet, starts the bot and the /health page
src/config.js	Reads all settings (.env / Render environment) and checks they make sense
src/core/orders.js	The business rules: statuses, allowed steps, fees, commission, phone and reference checks
src/core/orderService.js	Moves orders between statuses safely, one at a time, and checks who is allowed

File	What it does
src/core/eta.js	Delivery time estimates and learning a shop's cooking time
src/bot/customer.js	The conversation with customers (Part 3, steps 1–6)
src/bot/ops.js	Buttons for shops, riders and admins, and the admin commands
src/bot/reviews.js	Stars, thumbs and comments after delivery
src/bot/notify.js	Sending messages, and sending shop messages to the group when the shop has no Telegram
src/i18n.js	Every sentence the bot says, in Afaan Oromo, Amharic and English
src/storage/	Reading and writing the Google Sheet (and a memory-only version for demos)
scripts/setup-sheet.js	Creates the tabs and headers in a new sheet (npm run setup-sheet)
render.yaml, DEPLOY.md	How the bot is set up on Render

Words you will meet

Word	Meaning
Bot token	The bot's secret password. Anyone with it can control the bot. If it leaks: @BotFather → /revoke
Long polling	The bot asking Telegram "anything new?" over and over
Deploy	Putting a new version of the code onto Render
Logs	The bot's diary on Render (Dashboard → service → Logs). Look here first when something is wrong
Status	Where an order is in its life: PLACED, ACCEPTED, … DELIVERED
Callback / button	The buttons under bot messages. Each press is checked: who pressed it, and is it allowed now
Service account	A Google "robot user" that the bot uses to open your sheet. The sheet must be shared with it